Solidity hands-on lesson

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Arnaldo Tomasino

Gabriele Colapinto

April 16, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 Introduction **5**

2 Understanding the Development Toolkit **7**
2.1 Core Environment: Hardhat 7
2.2 Essential Prerequisites 8

3 Project Initialization and Structure **9**
3.1 Initializing a Hardhat Project 9
3.2 Anatomy of a Hardhat Project 9

4 Dissecting the Purchase.sol Smart Contract **13**
4.1 Core Logic and State Machine 13
4.2 Key Functions and Modifiers 13
4.3 Clarifications about the code 14

5 The Core Workflow: Compiling, Testing, and Deploying **15**
5.1 Compilation and Testing 15
5.2 Running a Local Network 15
5.3 Deploying the Contract 17

6 Interacting with the Deployed Contract via Remix **19**
6.1 Connecting Remix to the Local Hardhat Node 19
6.1.1 Using MetaMask as an injected provider 19
6.1.2 Using an external HTTP provider 21
6.1.3 Compiling and deploying the contract 22
6.2 Calling Contract Functions 22

7 Conclusion **27**

1 Introduction

This document serves as a practical, step-by-step guide for developers navigating the lifecycle of a Solidity smart contract. We will walk through the entire process, from initial project setup and coding to comprehensive testing, network deployment, and finally, interaction with the live contract. Our primary tools for this journey will be the **Hardhat** development environment and the **MetaMask** browser wallet. The project will center around building and deploying a "Safe Remote Purchase" contract, a common escrow pattern that demonstrates the power of smart contracts in facilitating trustless transactions.

2 Understanding the Development Toolkit

A robust development environment is strategically vital in the world of blockchain. Tools like Hardhat provide the essential scaffolding for writing, testing, and deploying secure and efficient smart contracts. These tools abstract away low-level network complexities like consensus mechanisms, allowing developers to focus on application logic and security rather than the intricacies of the underlying Ethereum network.

2.1 Core Environment: Hardhat

Hardhat is a professional-grade Ethereum development environment designed to streamline the entire smart contract lifecycle. It equips developers with a comprehensive suite of tools to manage projects of any scale, from simple proofs-of-concept to large-scale decentralized applications.

Its key capabilities include:

- **Development:** A complete framework for writing, debugging, and deploying Solidity contracts.
- **Testing:** A comprehensive testing approach that supports both fast, concise unit tests written directly in Solidity and more expressive, complex integration tests written in TypeScript. This process is accelerated by a high-performance runtime implemented in Rust.
- **Network Simulation:** The ability to run, test, and debug contracts on various networks, including simulated local nodes for rapid iteration, public test networks like Sepolia for staging, and production mainnets for final deployment.
- **Deployment System:** The integrated **Hardhat Ignition** module provides a simple, reliable, and declarative system for defining and executing contract deployments, ensuring a predictable and repeatable process.

2.2 Essential Prerequisites

Before beginning, you will need a few essential tools installed and configured on your development machine. These form the foundation upon which the Hardhat environment operates.

Tool	**Purpose**
Node.js (version 18 or higher) & npm	The JavaScript runtime and package manager required to install and execute Hardhat and its related dependencies.
Text Editor (e.g., Visual Studio Code)	The environment for writing Solidity and TypeScript code. Editors like VS Code offer official Solidity extensions that provide valuable syntax highlighting and analysis.
MetaMask	A browser extension that functions as a crypto wallet. It is used to manage accounts, sign transactions, and interact with dApps and deployed contracts on any network.

With this toolkit configured, we can proceed to initialize the project structure.

3 Project Initialization and Structure

A well-organized project structure is fundamental for creating maintainable and scalable smart contract applications. Hardhat establishes a logical and intuitive folder structure by default, which clearly separates key concerns such as contract source code, tests, and deployment scripts.

3.1 Initializing a Hardhat Project

The first step is to create an empty folder for the project. Once inside that directory, you can initialize a new Hardhat project using a single command in your terminal:

```
npx hardhat --init
```

This command will guide you through a setup process, after which it will generate the necessary configuration files and directory structure (See figure 3.1).

During the setup process choose the following options:

1. Choose Hardhat version 2
2. Keep the "." (current directory) as the project directory
3. As the project type, choose the Typescript one using Mocha and Ether
4. Install the required dependencies
5. Ignore the vulnerabilities because they are not important to the ends of this hands-on lesson

3.2 Anatomy of a Hardhat Project

The initialization process creates several key directories and files, each with a distinct purpose:

- `contracts:` This folder is the designated location for all Solidity smart contract source files (files with a `.sol` extension). This is where the core logic of your application will reside.
- `ignition:` This folder contains TypeScript scripts that leverage the **Hardhat Ignition** module. These scripts define the deployment logic for your smart contracts in a declarative way, ensuring that the process of deploying to any network is simple and reliable.
- `node_modules:` It is the folder containing all project dependencies — that is, the libraries and packages you have installed (Hardhat, ethers.js, Mocha, TypeScript, etc.). It is generated automatically when you run `npm install`. It is very large (contains thousands of files) and should never be committed to Git (it is usually in `.gitignore`).

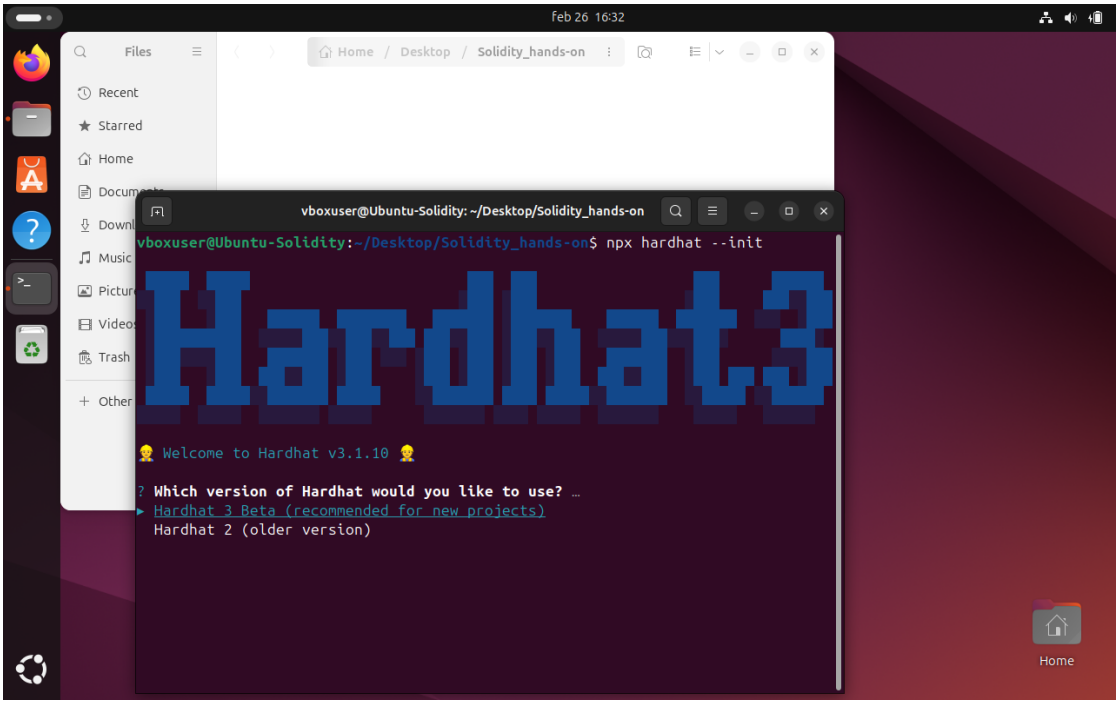

Figure 3.1: Hardhat initialization

- `test:` This directory holds your validation logic. It supports two distinct testing philosophies: fast, internal unit tests written in Solidity (often ending in `.t.sol`) that execute directly within the EVM, and external integration tests written in TypeScript/JavaScript that interact with the contract from the outside, simulating how a front-end or other service would.
- `hardhat.config.js` (or `.ts`) This is the central configuration file for the project. It is where you define network details, such as connections to a local simulated node, public testnets like Sepolia, or the production Ethereum mainnet. It also configures the Solidity compiler version and other project-wide settings.
- `package.json:` It is the main configuration file of your project. It contains important metadata such as the project name, version, required dependencies (both direct and development), and npm scripts you can run. When you share the project with others, they will have this file and can recreate the entire environment by running `npm install`.
- `package-lock.json:` It is a file automatically generated by npm that "freezes" the exact versions of all installed dependencies (including sub-dependencies). It ensures that anyone who clones the project obtains exactly the same versions, avoiding incompatibilities. It is important to commit it to Git along with `package.json`.
- `README.md:` It is a documentation file in Markdown format that describes your project: what it does, how to install it, how to use it, etc. It is the first file others read when they see your repository. Hardhat generates a base template that you can customize.
- `tsconfig.json:` It is the TypeScript configuration file. It specifies how the TypeScript compiler should behave: which JavaScript version to target, which modules to use, where

to look for source files, etc. It allows TypeScript to correctly convert your `.ts` code into executable JavaScript.

With the project structure in place, we can now focus on the specific smart contract we will be building.

4 Dissecting the Purchase.sol Smart Contract

The example contract we will build, `Purchase.sol`, implements a "Safe Remote Purchase" pattern. This is a common and practical use case for smart contracts, where an agreement is enforced by code to create trust between a buyer and a seller without relying on a traditional third-party intermediary. The contract acts as a digital escrow, holding funds until specific conditions are met by both parties.

To proceed with the hands-on lesson go to the "contracts" folder, create a file, call it "Purchase.sol", copy the code from the "Safe Remote Purchase" section of the official Solidity website and paste it in the newly created file.

4.1 Core Logic and State Machine

The contract's core function is to manage an escrow process. To initiate a sale, a seller deploys the contract and deposits funds equal to **twice the item's value**. To proceed, a buyer must also deposit funds equal to **twice the item's value**. These funds remain locked within the contract until the buyer explicitly confirms they have received the item. This mechanism ensures both parties have significant "skin in the game", incentivizing them to complete the transaction honestly.

The contract's logic is governed by a simple state machine defined by the `State` enum:

1. `Created`: The initial state after the seller deploys the contract and deposits their escrow. The contract is awaiting a buyer.

2. `Locked`: The state after a buyer has called `confirmPurchase()` and deposited their corresponding escrow. Both parties' funds are now locked.

3. `Release`: The state after the buyer has called `confirmReceived()`, signaling the item has been delivered. Funds are now ready for distribution.

4. `Inactive`: The final state after the transaction is successfully completed and all funds have been paid out, or after the transaction has been aborted.

4.2 Key Functions and Modifiers

The contract's behavior is exposed through several public functions, with access controlled by modifiers that restrict who can call them and in what state.

Function/Modifier	Role	Action
`constructor()`	Seller	Deploys the contract, sets the item's `value`, and locks the seller's initial deposit (which is 2x the `value`).
`onlySeller/onlyBuyer`	System	These modifiers restrict function execution, ensuring that only the designated seller or buyer can perform certain critical actions.
`confirmPurchase()`	Buyer	Deposits funds equal to `2 * value` to lock in the purchase and transitions the contract's state to `Locked`.
`confirmReceived()`	Buyer	Confirms the item was delivered, transitions the state to `Release`, and triggers a refund of `value` (half of the buyer's total deposit) back to the buyer.
`refundSeller()`	Seller	Called after the buyer's confirmation. It pays the seller `3 * value` (their original `2 * value` deposit plus the `value` of the item) and sets the state to `Inactive`.
`abort()`	Seller	Can only be called in the `Created` state. This cancels the sale, refunds the seller's full deposit, and sets the state to `Inactive`.

4.3 Clarifications about the code

In the body of the modifiers we can see the following command: `"_;"`. This is a placeholder for the body of the functions to which the modifier will be applied. This means that modifiers can be used as preconditions or post-conditions to the other functions depending on the cases. This makes Solidity more flexible and less error prone.

In the body of the constructor we can see that the value of the item is set as the half of the message value and then the code checks if the message value is an even number by checking that the double of the item value is equal to the message value. This process may seem unnecessary but it follows naturally from the need to first compute the item value and then verify that no precision was lost in the integer division, making the check a logical consequence of the assignment rather than an independent precondition. Basically, if the message value was an odd number the result of the division by 2 would have been the floor approximation of the actual result and doubling the item value wouldn't have led to finding the message value.
One could have also checked the parity of the message value before performing the division using a modulo operator (`msg.value % 2 == 0`) or the bitwise `&` for higher performances (`msg.value & 1 == 0`) and would have achieved the same result.

Now that we understand the contract's code, we can move on to the practical steps of compiling, testing, and deploying it.

5 The Core Workflow: Compiling, Testing, and Deploying

The smart contract development lifecycle follows an essential, sequential path. Once the Solidity code is written, it must be compiled into Ethereum Virtual Machine (EVM) bytecode, rigorously tested to identify and eliminate bugs, and finally deployed to a network to become operational. Hardhat streamlines each of these critical stages with simple command-line instructions.

5.1 Compilation and Testing

First, the entire project, including all contracts and their dependencies, can be compiled with a single command:

```
npx hardhat compile
```

After a successful compilation, you can run all associated tests using:

```
npx hardhat test
```

This command automatically discovers and executes all test files within the `test/` directory, running both Solidity-based unit tests and TypeScript-based integration tests. This unified test runner is a cornerstone of the Hardhat workflow, ensuring that both high-level behavior and low-level contract logic are validated before any deployment is considered. Never deploy untested code.

How to write tests is not part of the syllabus of this course but you can generate a TypeScript tests file using AI, put it in the `test` folder and run the tests using the aforementioned command. You will see an output similar to the one depicted in figure 5.1.

5.2 Running a Local Network

For rapid development and testing, Hardhat allows you to run a simulated Ethereum network directly on your local machine. To start this network, use the command:

```
npx hardhat node
```

This local node provides a complete, sandboxed blockchain environment that includes:

- A JSON-RPC server that perfectly mimics a real Ethereum network, allowing tools like MetaMask to connect to it.
- A set of pre-generated, externally owned test accounts.
- A starting balance of 10,000 ”fake” ETH for each test account, which can be used to pay for gas fees during development and testing without any real-world cost.

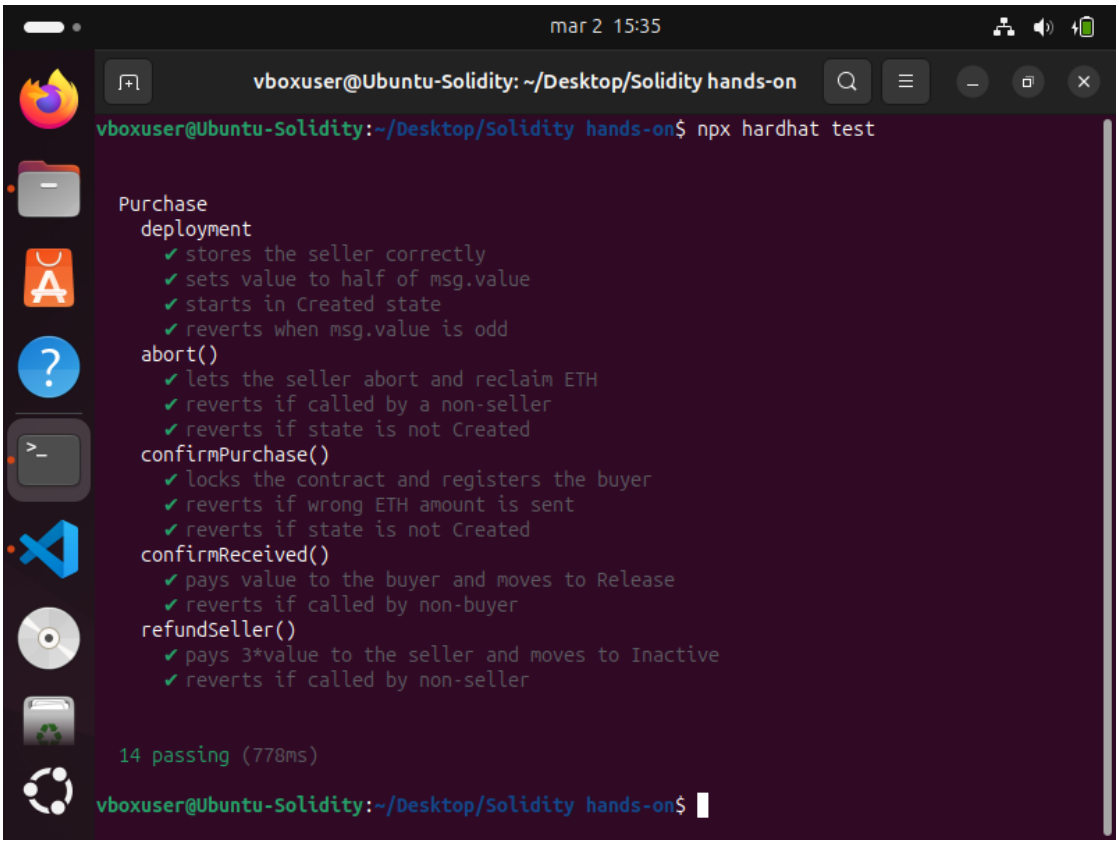

Figure 5.1: Running tests automatically in Hardhat

5.3 Deploying the Contract

With a local network running, the next step is to deploy the smart contract. This process is managed by the scripts located in the `ignition/` folder. To execute a deployment script, you specify the script's path and the target network:

```
npx hardhat ignition deploy \
ignition/modules/Purchase.ts --network localhost
```

How to create an ignition deployment script is not part of the syllabus of this course but you can generate it using AI, call it `Purchase.ts` and put it in the `ignition/modules` folder.

A successful deployment will output the address of the newly created smart contract on the local blockchain. This address is the unique identifier for your contract instance and is essential for the final phase: interaction.

With the contract compiled, tested, and deployed, it is now live on our local network and ready to be interacted with.

6 Interacting with the Deployed Contract via Remix

While command-line tools are powerful, a graphical user interface (GUI) can provide a more intuitive way to interact with a deployed contract and verify its functionality. The Remix IDE, an online Solidity development environment, is an excellent tool for this purpose. We can connect it to our local Hardhat node to call contract functions and observe state changes in real-time.

6.1 Connecting Remix to the Local Hardhat Node

This process effectively uses Remix as a powerful GUI to "attach" to a contract managed by our terminal-based Hardhat environment, a common pattern for quick, visual interaction without leaving your primary workflow.

There are two equivalent ways to connect Remix IDE to our local Hardhat environment: using an injected provider like MetaMask or by using an external HTTP provider.

6.1.1 Using MetaMask as an injected provider

Before we begin with the procedure note that some versions of MetaMask do not allow us to use MetaMask as an intermediary between our local Hardhat environment and the Remix IDE. If that's the case, it won't be possible to run transactions through MetaMask but we can still use the latter to monitor the balances of the accounts we use for the transactions using the other method.

1. Set up MetaMask and connect it to the local hardhat network following this guide: Connecting MetaMask with a Local Hardhat Network.
2. Select an imported account (see figure 6.1).
3. Keep the MetaMask tab open in the browser and open a new tab.
4. In the "Environment" dropdown, change the selection from "Remix VM" to an "Injected Provider" (like MetaMask). Your MetaMask wallet should be configured to connect to your local Hardhat network (e.g., `localhost:8545`) if you followed the guide correctly. After you select this dropdown option you will see the inteface depicted in figure 6.2. The account you see in the interface is the one you selected in the MetaMask tab of your browser.
5. At this point you can see the selected account in the "ACCOUNT" dropdown menu and, if the balance of the account is 0 Ether, it means that the current version of MetaMask does not support this operation (see figure 6.3).

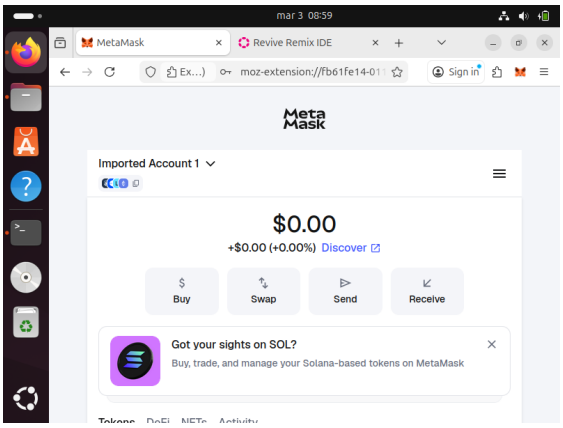

Figure 6.1: Select an imported account in MetaMask

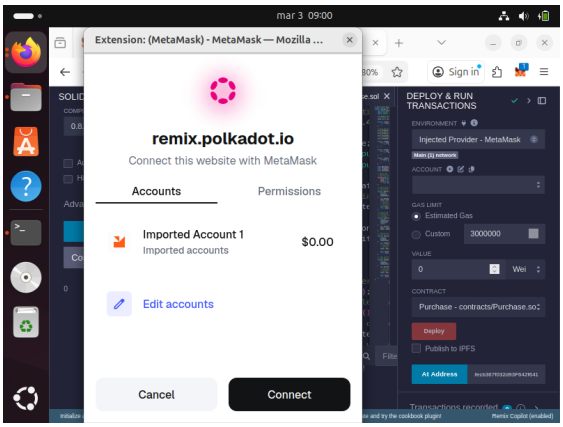

Figure 6.2: Injected provider selection

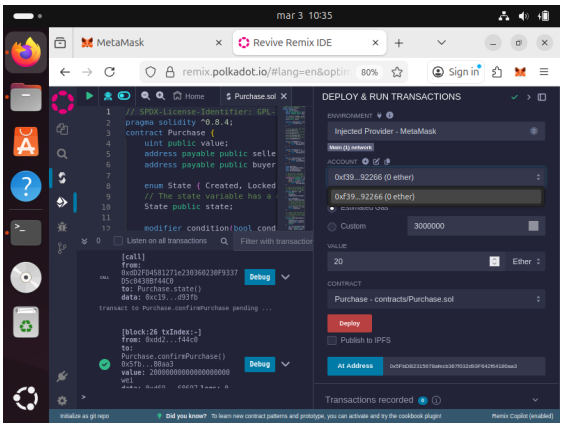

Figure 6.3: The account has a balance of 0 Ether because the current version of MetaMask does not support the procedure

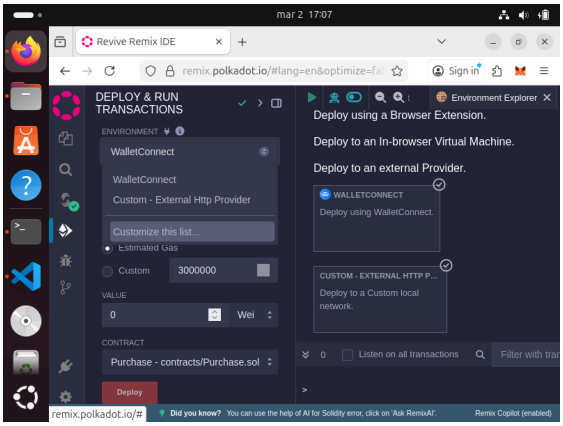

Figure 6.4: Adding an environment in Remix IDE

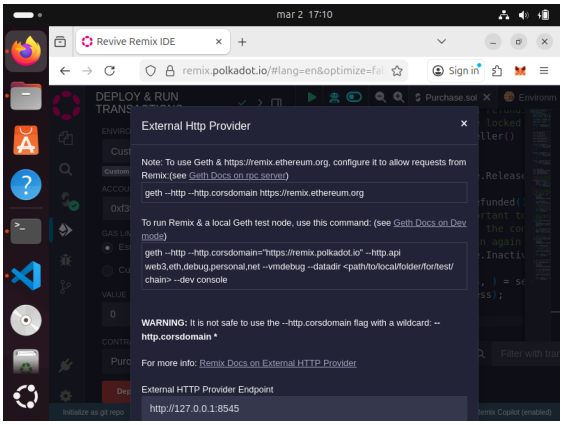

Figure 6.5: External HTTP provider popup

6.1.2 Using an external HTTP provider

Using an external HTTP provider directly connects the Remix IDE to our local Hardhat environment.

1. In the Remix IDE go to the “Deploy & Run Transactions” tab on the sidebar on the left, click the ”ENVIRONMENT” dropdown menu and select ”Customize this list...”.
2. In the menu, search for the option that says ”Deploy to a custom local network” and tick the mark in the top-right corner of the box (see figure 6.4).
3. Now we should be able to see and select the ”Custom - External Http Provider” option in the environment dropdown.
4. When the popup in figure 6.5 shows up, press Enter.
5. Now, in the account dropdown we should be able to select one of the many accounts we have created using the `npx hardhat node` command and select one (see figure 6.6).

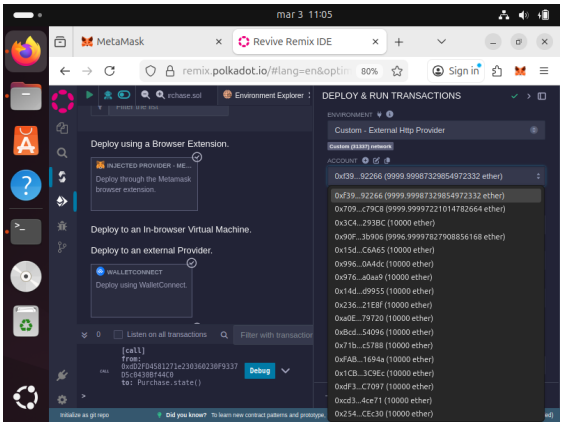

Figure 6.6: Local Hardhat accounts list

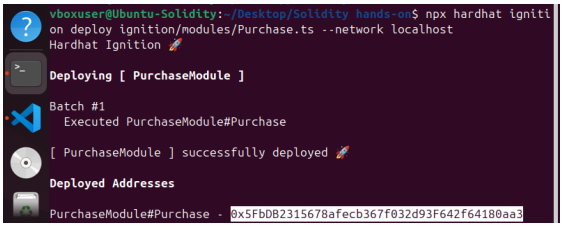

Figure 6.7: Contract address generated during the deployment phase

6.1.3 Compiling and deploying the contract

1. Go to the Remix IDE, create a new file (e.g., `Purchase.sol`) and paste the exact source code of your smart contract into it.

2. Compile the contract within Remix. This step does not deploy a new instance; instead, it provides Remix with the contract's Application Binary Interface (ABI), which is the "manual" for how to interact with its functions.

3. Navigate to the "Deploy & Run Transactions" tab on the sidebar on the left in Remix.

4. Instead of using the "Deploy" button, find the "At Address" field. Paste the contract address you obtained from the Hardhat deployment step (see figure 6.7) into this field and click the button at its side (see figure 6.8).

6.2 Calling Contract Functions

After successfully connecting Remix to the contract address, navigate to the bottom of the column and click the arrow at the left end of the deployed contract (see figure 6.9). The IDE will display a user interface with clickable buttons for all the public functions of the `Purchase` contract (see figure 6.10).

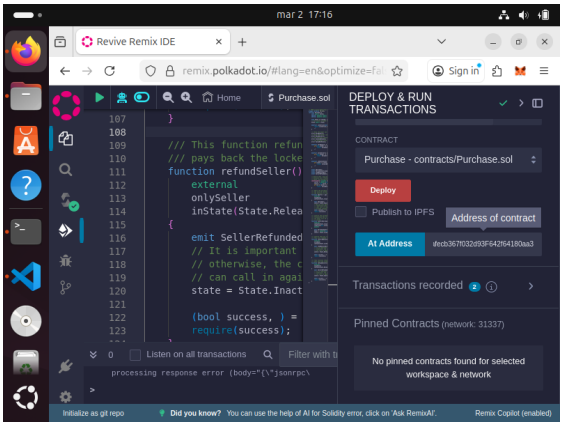

Figure 6.8: Insertion of the contract address

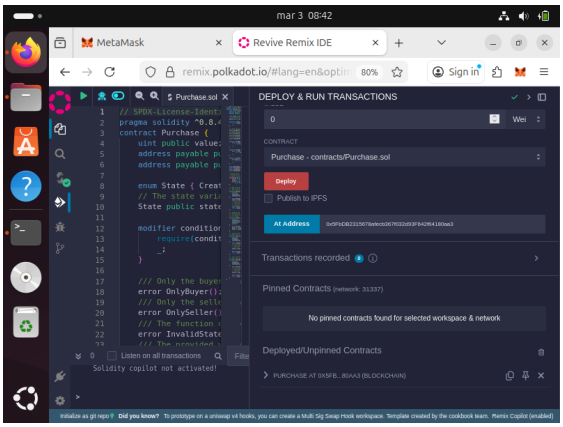

Figure 6.9: Deployed contract

You can now call functions on the contract using the buttons on the interface. For `payable` functions like `confirmPurchase`, you must also specify the amount of Ether you wish to send with the transaction in the input field right above the contract field (see figure 6.11). After calling a function, we can see its results in the Remix IDE terminal in the bottom-left end of the screen (see figure 6.12) and in the Hardhat node console on our local machine (see figure 6.13).

Now that we issued the transaction, we can see that the value of the transaction was deducted from the account using both the Remix IDE (see figure 6.14) and MetaMask (see figure 6.15).

This end-to-end process allows for complete validation of the contract's logic and behavior in a controlled, interactive environment.

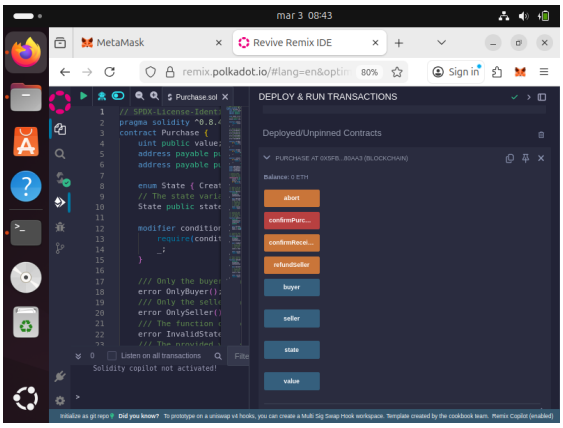

Figure 6.10: Deployed contract functions

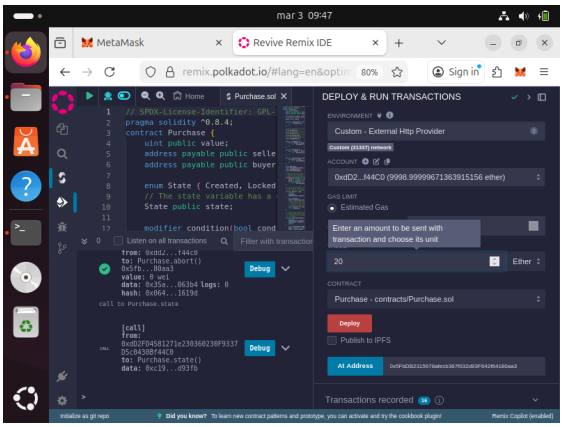

Figure 6.11: Enter the transaction value

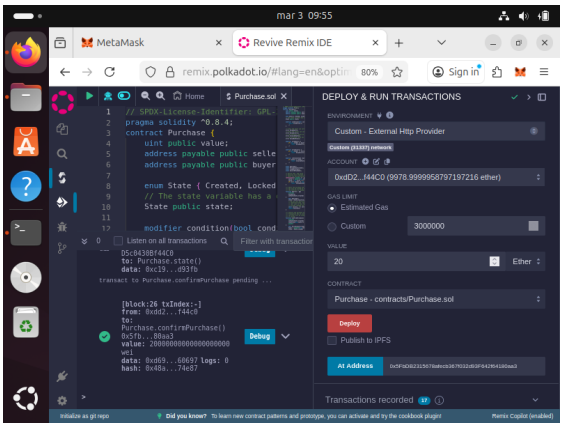

Figure 6.12: Transaction result in the Remix IDE - Notice that the value of 20 Ether was converted in Wei

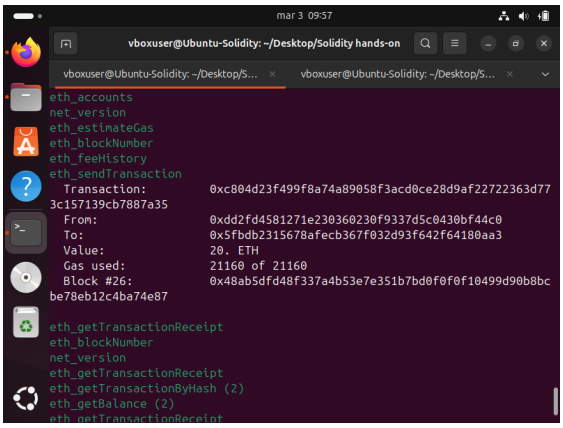

Figure 6.13: Transaction result in the Hardhat node console on our local machine

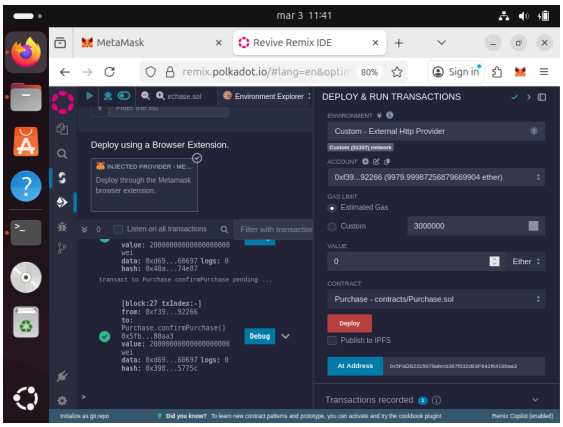

Figure 6.14: New balance of the account in the Remix IDE

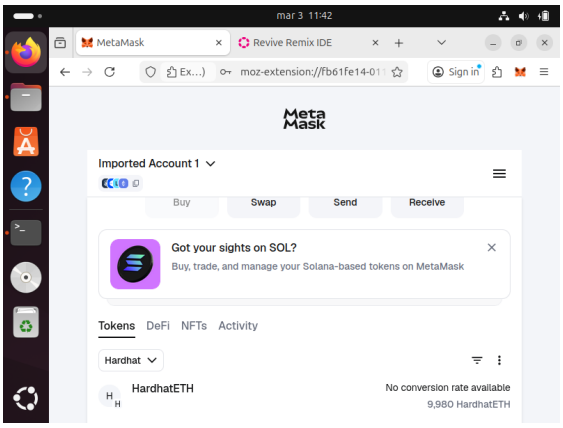

Figure 6.15: New balance of the account in MetaMask - In this case the number is rounded

7 Conclusion

This guide has walked through the complete, end-to-end workflow of modern smart contract development. We covered setting up a professional development environment with Hardhat, organizing a project with its logical structure, writing a stateful Solidity contract for a real-world use case, and then systematically compiling, testing, deploying, and interacting with it. By leveraging industry-standard tools like Hardhat for the backend lifecycle and MetaMask and Remix for frontend interaction, developers can build, validate, and deploy robust decentralized applications with confidence. Mastering this workflow is a fundamental competency for any developer looking to build on the blockchain.